

SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192311097
Name	DILLILOKESH D

COURSE OUTCOME COVERED

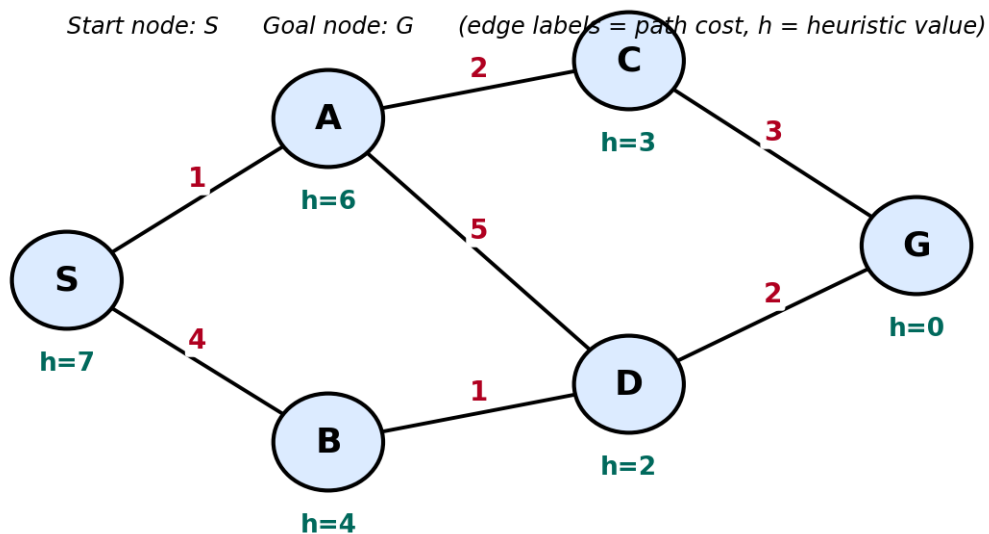
CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

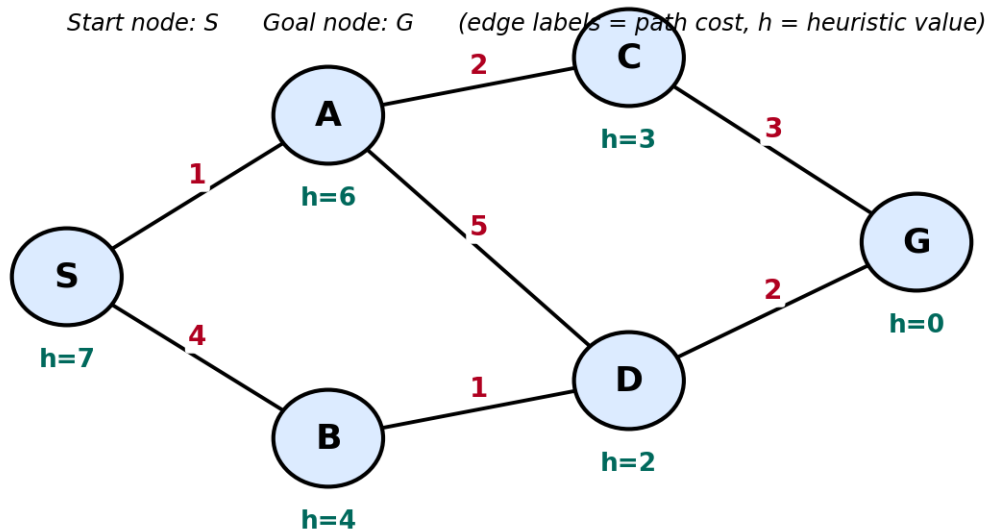
Total Marks:50

Q1. Consider a suitable state-space graph (with heuristic values $h(n)$ assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.



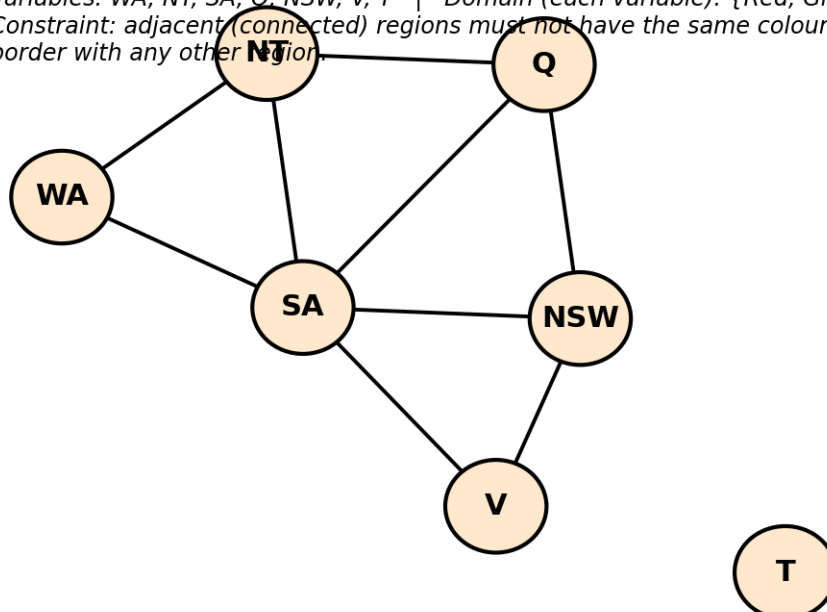
Q2. For a suitable weighted state-space graph (with edge costs and heuristic values $h(n)$) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the $g(n)$, $h(n)$ and $f(n)$ values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.



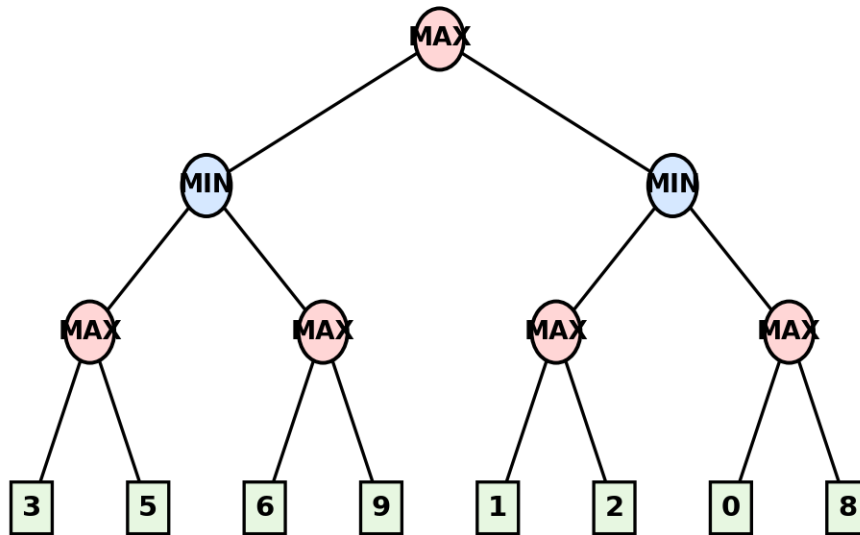
Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.

Variables: WA, NT, SA, Q, NSW, V, T | Domain (each variable): {Red, Green, Blue}
 Constraint: adjacent (connected) regions must not have the same colour. T has no shared border with any other region.



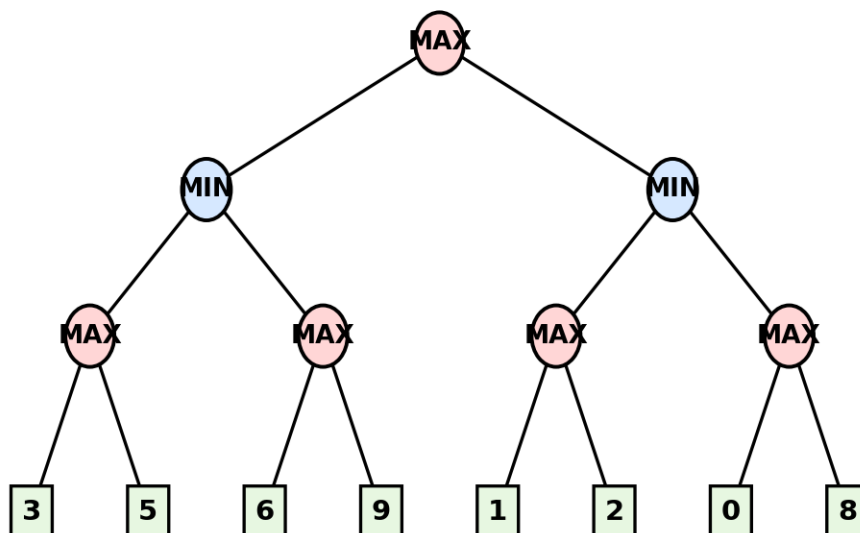
Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

I DILLILOKESH D 192311097 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Dry Run Evaluation**RUBRICS FOR CALCULATION**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	10
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	10
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α - β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the final outcome.	Multiple calculation errors are present that affect the correctness of intermediate steps.	Calculations are mostly incorrect or missing.	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	10
Clarity of Presentation & Justification (trace tables/trees/diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	10
				Total	50

ANSWERS

1. Greedy Best-First Search

Pseudocode:

- Insert start node into frontier, ordered by $h(n)$.
- Loop: if frontier is empty, return failure.
- Remove the node with lowest $h(n)$ from frontier; call it current.
- If current is the goal, return the path.
- Otherwise, expand current: generate its neighbours, add unvisited ones to the frontier with their $h(n)$ values.
- Mark current as visited/expanded and repeat.

Properties:

- Completeness: Not complete in general graphs (can loop) unless duplicate checking is used; complete on finite graphs with visited tracking.
- Optimality: Not optimal — it can settle for a longer path if the heuristic makes a wrong node look closer, as seen in this very problem.
- Time Complexity: $O(b^m)$ in the worst case, where b is branching factor and m is maximum depth, though a good heuristic prunes this drastically in practice.
- Space Complexity: $O(b^m)$, since all generated nodes are kept in the frontier.

Graph used: Start = S ($h=7$), Goal = G ($h=0$)

Nodes: A($h=6$), B($h=4$), C($h=3$), D($h=2$), G($h=0$)

Edges (cost): S-A=1, S-B=4, A-C=2, A-D=5, B-D=1, C-G=3, D-G=2

Greedy Best-First Search always picks the node with the lowest $h(n)$ from the frontier, regardless of path cost so far.

Step	Node Expanded	Frontier (Open List) after expansion	h(n) used
1	S (start)	A(6), B(4)	h(S)=7
2	B	A(6), D(2)	h(B)=4
3	D	A(6), G(0)	h(D)=2
4	G (goal)	— search stops, goal found	h(G)=0

Path selected: $S \rightarrow B \rightarrow D \rightarrow G$

Total path cost = $4 + 1 + 2 = 7$

2. A* Search

Pseudocode:

- Insert start node into frontier with $f(\text{start}) = h(\text{start})$, since $g(\text{start}) = 0$.
- Loop: if frontier is empty, return failure.
- Remove the node with the lowest $f(n)$ from frontier; call it current.
- If current is the goal, return the reconstructed path and its cost.
- Otherwise, for each neighbour, compute $g(\text{neighbour}) = g(\text{current}) + \text{edge cost}$, and $f(\text{neighbour}) = g(\text{neighbour}) + h(\text{neighbour})$. Add or update it in the frontier.
- Mark current as expanded and repeat.

Properties:

- Completeness: Complete, provided the branching factor is finite and edge costs are bounded above zero.
- Optimality: Optimal as long as the heuristic $h(n)$ is admissible (never overestimates the true cost to the goal) — in this problem h is admissible, so A* is guaranteed to return the cheapest path.
- Time Complexity: $O(b^d)$ in the worst case, but typically far lower with a well-designed heuristic, where d is the depth of the optimal solution.
- Space Complexity: $O(b^d)$, since it keeps every generated node in memory — this is A*'s main drawback for very large search spaces.

Same weighted graph as Q1 is used, with $f(n) = g(n) + h(n)$.

Step	Node Expanded	$g(n)$	$h(n)$	$f(n)=g+h$	Frontier after this step
1	S	0	7	7	A($f=7$), B($f=8$)
2	A	1	6	7	C($f=6$), B($f=8$), D($f=8$)
3	C	3	3	6	G($f=6$), B($f=8$), D($f=8$)
4	G (goal)	6	0	6	goal reached, $f(G)$

Working:

- $g(A) = 1, f(A) = 1+6 = 7$
- $g(B) = 4, f(B) = 4+4 = 8$
- $g(C) = g(A)+2 = 3, f(C) = 3+3 = 6$
- $g(D \text{ via } A) = g(A)+5 = 6, f(D) = 6+2 = 8$
- $g(G \text{ via } C) = g(C)+3 = 6, f(G) = 6+0 = 6 \rightarrow$ lowest f in frontier, expand G, goal reached

Optimal path: $S \rightarrow A \rightarrow C \rightarrow G$

Total path cost = $1 + 2 + 3 = 6$

3. CSP – Map Colouring (Australia) with Backtracking Search

Pseudocode:

- If assignment is complete, return it as the solution.
- Select an unassigned variable (here, in the fixed order WA, NT, SA, Q, NSW, V, T).
- For each value in that variable's domain: if the value is consistent with the constraints so far, assign it and recursively call backtracking search on the rest of the variables.
- If the recursive call succeeds, return the solution.
- If it fails, remove the assignment (backtrack) and try the next value.
- If all values have been tried and none work, return failure to the previous variable.

Properties / Enhancements commonly discussed:

- Completeness: Backtracking is complete — it will find a solution if one exists, since it systematically explores the whole search tree.
- Variable/Value ordering heuristics: Minimum Remaining Values (MRV) and Degree Heuristic help choose which variable to assign next; Least Constraining Value helps choose which value

to try first — these can reduce backtracking substantially, though a fixed order was used in this dry run.

- Constraint propagation: techniques like forward checking and arc-consistency (AC-3) can detect failures early, before backtracking is even needed.

Variables: WA, NT, SA, Q, NSW, V, T

Domain (each variable): {Red, Green, Blue}

Constraint: adjacent (bordering) regions must not get the same colour. T shares no border with any other region, so it has no constraint linking it to the rest.

Adjacency used: WA–NT, WA–SA, NT–SA, NT–Q, SA–Q, SA–NSW, SA–V, Q–NSW, NSW–V

Variable order for backtracking: WA, NT, SA, Q, NSW, V, T

Step	Variable	Value Tried	Constraint Check	Result
1	WA	Red	No prior assignments to conflict with	Accepted
2	NT	Red → conflict with WA(Red); try Green	NT is adjacent to WA	Green accepted
3	SA	Red → conflicts WA; Green → conflicts NT; try Blue	SA adjacent to WA & NT	Blue accepted
4	Q	Red → check vs NT(Green), SA(Blue); no conflict	Q adjacent to NT & SA	Red accepted
5	NSW	Red → conflicts Q; Green → check vs SA(Blue); no conflict	NSW adjacent to SA & Q	Green accepted
6	V	Red → check vs SA(Blue), NSW(Green); no conflict	V adjacent to SA & NSW	Red accepted
7	T	Red	T has no adjacency constraint	Red accepted

Final consistent solution:

- WA = Red
- NT = Green
- SA = Blue
- Q = Red
- NSW = Green
- V = Red

- T = Red (unconstrained)

4. Minimax Algorithm

Pseudocode:

- If node is a terminal/leaf node, return its utility value.
- If it is MAX's turn at this node, return the maximum value of MINIMAX(child) over all children.
- If it is MIN's turn at this node, return the minimum value of MINIMAX(child) over all children.
- Recurse down to the leaves first, then propagate values back up level by level, exactly as done step-by-step below.

Properties:

- Completeness: Complete for finite game trees.
- Optimality: Optimal against a perfectly rational opponent who is also trying to optimise their own utility.
- Time Complexity: $O(b^d)$, where b is the branching factor (number of moves available) and d is the depth of the tree.
- Space Complexity: $O(b*d)$ for depth-first exploration, or $O(b^d)$ if the whole tree is stored.

Game tree (3 levels below root): Root = MAX, Level 2 = MIN (2 nodes), Level 3 = MAX (4 nodes), Level 4 = terminal leaves (utility values).

Leaf values (left to right): 3, 5, 6, 9, 1, 2, 0, 8

Step 1 — back up leaf values to the Level-3 MAX nodes (each MAX node takes the maximum of its two children):

MAX node	Children (leaves)	Backed-up value = MAX(children)
MAX1 (leftmost)	3, 5	5
MAX2	6, 9	9
MAX3	1, 2	2
MAX4 (rightmost)	0, 8	8

Step 2 — back up to the Level-2 MIN nodes (each MIN node takes the minimum of its two children):

MIN node	Children (MAX values)	Backed-up value = MIN(children)
MIN-Left	MAX1=5, MAX2=9	5
MIN-Right	MAX3=2, MAX4=8	2

Step 3 — back up to the Root (MAX takes the maximum of MIN-Left and MIN-Right):

Root value = $\text{MAX}(5, 2) = 5$

Best move for MAX: choose the LEFT subtree (MIN-Left, value 5), which is reached through MAX1, which in turn is achieved by the leaf valued 5.

Justification: the left subtree guarantees a payoff of 5 to MAX even under optimal (worst-case) play from MIN, whereas the right subtree only guarantees 2. So the left move dominates.

5. Minimax with Alpha-Beta Pruning

Pseudocode:

- ALPHABETA(node, α , β , player): if node is terminal, return its utility.
- If player is MAX: for each child, value = max(value, ALPHABETA(child, α , β , MIN)); update $\alpha = \max(\alpha, \text{value})$; if $\alpha \geq \beta$, stop evaluating remaining children (β cut-off).
- If player is MIN: for each child, value = min(value, ALPHABETA(child, α , β , MAX)); update $\beta = \min(\beta, \text{value})$; if $\beta \leq \alpha$, stop evaluating remaining children (α cut-off).
- Return the propagated value once all (non-pruned) children are evaluated.

Properties:

- Correctness: Always returns the same root value and best move as plain Minimax — pruning never changes the final decision, it only skips work that is provably irrelevant.
- Time Complexity: $O(b^d)$ worst case (no pruning at all), but with a good move ordering it can be as low as $O(b^{(d/2)})$, effectively doubling the search depth achievable in the same time compared to plain Minimax.
- Space Complexity: $O(b*d)$, same as depth-first Minimax.
- Order sensitivity: The amount of pruning achieved depends heavily on the order in which children are explored — exploring the best moves first leads to far more cut-offs, as seen with the MAX4 subtree below.

Traversal is left to right. α = best value MAX can guarantee so far, β = best value MIN can guarantee so far. A branch is pruned when $\alpha \geq \beta$.

Branches pruned:

- Leaf value 9 (right child of MAX2) — pruned because at that point $\alpha(6) \geq \beta(5)$ coming from

Node visited	α (before)	β (before)	Value computed	α / β updated	Pruning
Root (MAX)	$-\infty$	$+\infty$	—	$\alpha=-\infty, \beta=+\infty$	—
MIN-Left	$-\infty$	$+\infty$	—	—	—
MAX1	$-\infty$	$+\infty$	leaf 3 $\rightarrow v=3$, leaf 5 $\rightarrow v=5$	$\alpha=5$	none
(MIN-Left after MAX1)	$-\infty$	$+\infty$	MIN-Left tentative = 5	$\beta=5$	—
MAX2	$-\infty$	5	leaf 6 $\rightarrow v=6, \alpha=6$	$\alpha=6 \geq \beta=5$	leaf 9 PRUNED
MIN-Left (final)	$-\infty$	5	$\min(5,6)=5$	MIN-Left = 5	—
Root (after left)	5	$+\infty$	—	$\alpha=5$	—
MIN-Right	5	$+\infty$	—	—	—
MAX3	5	$+\infty$	leaf 1 $\rightarrow v=1$, leaf 2 $\rightarrow v=2$	α stays 5 ($2 < 5$)	none
MIN-Right (after MAX3)	5	$+\infty$	MIN-Right tentative = 2	$\beta=2$	$2 \leq \alpha=5$
MAX4	—	—	not evaluated	—	ENTIRE SUBTREE PRUNED (leaves 0, 8 never visited)
MIN-Right (final)	5	2	value = 2	MIN-Right = 2	—
Root (final)	5	2	$\max(5,2) = 5$	Root value = 5	—

MIN-Left, so MIN-Left could never prefer this branch over the 5 it already has.

- The whole MAX4 subtree (leaves 0 and 8) — pruned because $\beta(2)$ at MIN-Right already fell to/below the root's $\alpha(5)$, so MAX at the root would never pick this path anyway.

Final result: Root value = 5, best move = LEFT subtree — identical to plain Minimax in Q4, but Alpha-Beta reached it after evaluating fewer leaves (3, 5, 6, 1, 2 were checked; 9, 0, 8 were skipped).